2015 41st Euromicro Conference on Software Engineering and Advanced Applications

Adaptive Knowledge Bases in Self-Adaptive System Design

Verena Klös
Technische Universität Berlin
verena.kloes@tu-berlin.de

Thomas Göthel
Technische Universität Berlin
thomas.goethel@tu-berlin.de

Sabine Glesner
Technische Universität Berlin
sabine.glesner@tu-berlin.de

***Abstract*—Self-adaptive systems allow for flexible solutions in changing environments. Usually, a fixed set of predefined rules is used to define the adaptation possibilities of a system. The main problem of such systems is to cope with environment behaviours that were not anticipated at design-time. In this case, no adaptation rule might be applicable or adaptations might not have the expected effect. In this paper, we propose an extended architecture of IBM's MAPE-K loop to cope with this problem. We impose a structure on the knowledge base consisting of an abstract system and environment model, a global goal model, and a set of (current) adaptation rules. Furthermore, we introduce an evaluation component that deletes failed adaptation rules, as well as a learning component that uses run-time models to autonomously generate new rules if the current ones are not applicable. With our approach, not only functional components can dynamically be adapted but also the adaptation logic itself.**

I. Introduction

The aim of self-adaptive systems is to cope with changing environments autonomously. The key challenge is to adapt in a suitable way even if the range of possible environment changes is not known at design-time. In this case, it is likely that static adaptation mechanisms are not applicable or do not have the intended effect. This is especially the case in the context of cyber-physical systems (CPS) where the environment is highly dynamic and becomes even more likely if furthermore components can be added or removed at run-time (e.g., when a new air conditioner is installed in a temperature control system). To cope with dynamic changes of the environment, cyber-physical systems should have the ability to adapt their adaptation logic as well. However, it is a challenge to detect and remove inapplicable or inefficient adaptation mechanisms as well as to infer new ones that can cope with the new situation at run-time.

To control the adaptation process in a self-adaptive system, a common approach is to use an implementation of IBM's MAPE-K feedback loop [1], which autonomously Monitors and Analyses a managed system and Plans and Executes an adaptation plan if necessary. All parts of the MAPE-K loop share a common Knowledge base to exchange information.

In this paper, we present an extended architecture of the MAPE-K loop as a reference model for the design of self-adaptive systems that are able to dynamically adapt their adaptation mechanisms. For brevity, we here assume that a cyber-physical system has a central controller with a central MAPE-K loop. We propose to continuously evaluate adaptation steps concerning their actual effect and adaptation mechanisms concerning their applicability and efficiency in the case of topology changes. Inefficient or inapplicable adaptation rules are disabled or removed from the current set of available rules. To infer new adaptation rules, we propose simplified reinforcement learning techniques on executable run-time models of the system components and their interplay with the environment. These models are used to infer interrelations between system actions and subsequent environment behaviour. To cope with dynamic changes to the system topology, we assume that each system component provides its own run-time model as well as a set of associated mutation operations that can be used for learning. This also enables the integration of heterogeneous components from various manufacturers.

To realise the ideas above, we integrate an evaluation and a learning component and furthermore impose a dedicated structure on the knowledge base within the MAPE-K loop. This imposed structure consists of an abstract environment model, an abstract system model, and an abstract goal model. Furthermore, we let the adaptation mechanisms (in terms of adaptation rules) be part of the knowledge base to emphasize that this set can change at run-time. This structure provides a separation of knowledge concerns, which enables the interchangeability of these models.

The rest of this paper is structured as follows. In Section II, we discuss related work. Then, we present our proposed reference model in Section III and describe the knowledge base, the evaluation component, and the learning component. Furthermore, we discuss how Communicating Sequential Processes could be used as run-time model within the learning component and how formal analyses could be integrated in our approach. We illustrate the benefits of our approach using an example in Section IV. Finally, in Section V, we give a conclusion and point out possible directions for future work.

II. Related Work

In this section, we discuss related work. We particularly focus on general frameworks for self-adaptive systems.

In [2], an adaptive system is abstractly modelled as a collection of communicating services that can be reconfigured based on locally available data flow information. The adaptation behaviour of each service is described using configuration rules. The abstract model is also used for formal analysis. In contrast to our work, the authors consider adaptive systems only on

978-1-4673-7585-6/15 $31.00 © 2015 IEEE
DOI 10.1109/SEAA.2015.48

an abstract level to perform verification. This level is similar to the run-time model level in our approach. However, we focus on keeping the implementation and the abstract models consistent and on making adaptation mechanisms adaptive.

The work in [3] provides a model-based development approach for adaptive embedded systems in which adaptation behaviour is strictly separated from functional behaviour. Adaptation is realised by switching predefined configurations. In contrast to that work, we focus on more flexible adaptation mechanisms that are adaptive themselves.

In [4], the process calculus *Communicating Sequential Processes* (CSP) [12] is used to model self-adaptive applications. The current knowledge of a node containing simple behavioural rules such as abstract descriptions of communication protocols can be transferred to other nodes if necessary. In contrast, we are interested in more flexible solutions where previously not present rules can be learned.

In [5], a UML-based modelling language for adaptive systems that uses the concept of the MAPE-K loop is presented. The knowledge base contains additional accumulated and temporal information for analysis and planning, which allows for sophisticated adaptation mechanisms. However, these mechanisms cannot change over time dynamically.

In [6] and [7], generic reference models for adaptive systems using models@run.time are proposed. Models@run.time or run-time models are abstract representations of the running system and the current environment behaviour. In self-adaptive systems, they are used to plan adaptations and to estimate their effect. The reference models describe the interplay between environment, core functionality, and adaptation logic.

In [6], the reference model distinguishes between a ground level that basically describes a MAPE-loop, and higher levels of abstraction that introduce various run-time models. As a result, it allows for the structured design of adaptive systems and focusses on architectural mechanisms to base adaptation on. However, this reference model is very abstract and does not specify how the run-time models are used for adaptation and how adaptation mechanisms could be adapted. In contrast, we use abstract models as information for the MAPE components and only use more concrete run-time models inside the learning component. Furthermore, we impose a structure on the knowledge base and introduce additional components to manipulate the adaptation rules to handle unanticipated environmental behaviours.

In [7], an abstract approach is presented where adaptation is based on system models, a feedback loop, and goal models. Their reference model possesses similarities in the structure of the system models compared to our knowledge models and a reasoner that performs search-based learning on the system models. However, the general adaptation process within this reference model is given in a rather abstract way, especially the role of the plan models is not obvious. In contrast, we describe the general adaptation process within our reference model and provide details on our proposed knowledge models. We explicitly distinguish abstract system data and environment data that we use for rule-based planning from more concrete and executable run-time models used for search-based learning. Furthermore, we propose an evaluation of the effect of former adaptations based on a comparison of the expected and actually observed effect. Additionally, we illustrate how our approach can be used in an example scenario.

One approach that uses formal models not only for representing the knowledge but also for the other parts of the MAPE-K loop is ActivFORMs [8]. The authors propose to use UPPAAL timed automata [9] to model application specific MAPE-K components and a virtual machine to execute these automata. Furthermore, they introduce a goal management that adapts the MAPE models according to a goal model that can be updated by a system admin. The adaptation is realised as a switch between associated adaptation models for each goal. We, in contrast, propose a generic reference model that can be instantiated for an application by defining the abstract knowledge models and the control data that is exchanged between the functional components and the adaptation component. Our adaptation of the adaptation logic is realised with online learning techniques on run-time models to infer new adaptation rules without manual intervention.

Previously, we presented a formal pattern for the construction and modular verification of distributed adaptive real-time systems in [10]. There, we also followed the MAPE-K loop, but focussed on the separation of functional and adaptation components and how this can be exploited for analysis using the process calculus *Communicating Sequential Processes* (CSP) [12]. Furthermore, we discussed the possible application in the context of SystemC [11], which is a language that is well-suited for the design and implementation of communication-intensive hardware/software systems. In contrast, we here focus on the detailed design of the different parts of the MAPE loop and especially the knowledge base. Additionally, we present an abstract learning and evaluation component and sketch how they enable the adaptation of the adaptation logic. Some of our previous formal considerations can be reused but need to be significantly enhanced to be applicable in this flexible context.

III. Adaptive Adaptation Mechanisms

In this section, we present our proposed reference model for adaptive systems with adaptive knowledge. It extends IBM's MAPE-K loop [1] by an additional evaluation component and a learning component (see Fig. 1). Together with a dedicated structure of the knowledge base, not only functional components but also the adaptation mechanisms themselves can be adapted. Our abstract knowledge models are given by an environment model K_{Env}, a system model K_{Sys}, a goal model K_{Goal}, and an adaptation model (adaptation rules) K_{Adapt}. In the following, we first explain the general adaptation process within our reference model (depicted in Figure 2) before we go into the details of the different parts of our reference model.

In the monitoring phase, the adaptation component retrieves run-time information from the functional component and the environment. This information is used to update the abstract system model K_{Sys} and the abstract environment model

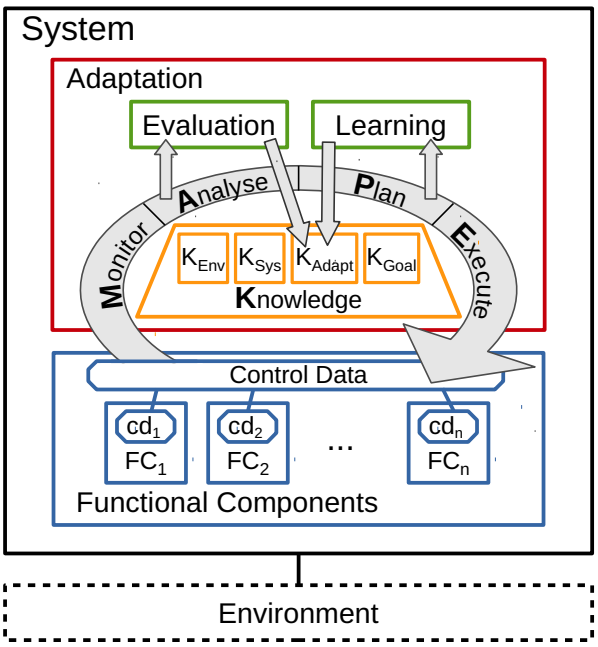

Fig. 1. Extended MAPE-K Loop

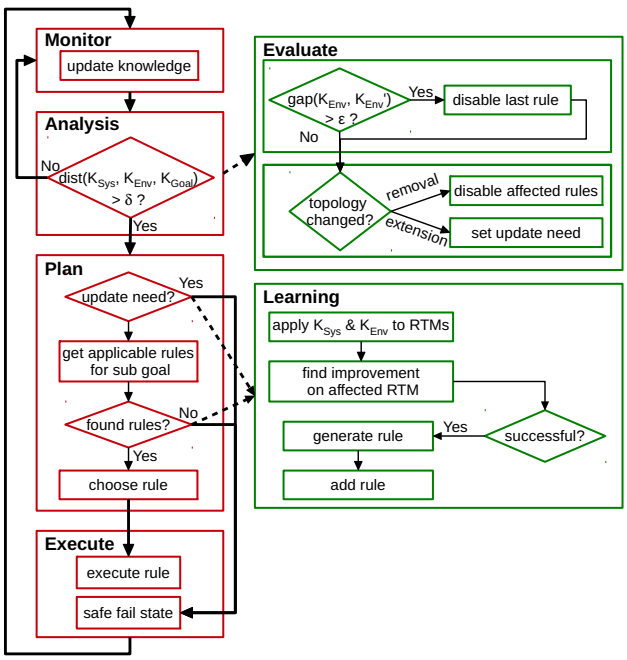

Fig. 2. Adaptation Process in Detail

K_{Env}. In the analysis phase, these models are used to decide whether adaptation is necessary or not. The decision is based on the "distance" of the system to the goal model K_{Goal}. If this distance, which is measured by a distance function that we assume to be provided by the goal model, exceeds a certain threshold δ, adaptation rules have to be applied to approach the goals again. Furthermore, the evaluation component is invoked in the analysis phase. It evaluates whether previous adaptations have been successful, i.e., whether the gap between the environment K_{Env} and the expected environment K'_{Env} is smaller than a tolerance value ε. If the gap is greater than ε, the corresponding rules are removed or disabled from the set K_{Adapt}, because they do not cope with the current state of the environment. In the case of dynamic changes to the system topology the evaluation component also disables adaptation rules that belong to a removed or disabled component and detects the need to update the rules due to new functionality provided by a newly added component. If an adaptation is necessary, the best available rule reestablishing the violated subgoal is chosen in the planning phase. If no suitable rule is available (due to an unforeseen situation) or if the rules need to be updated (due to topology changes), the learning component is invoked to infer a new rule. Finally, if a suitable adaptation rule could be found, it is applied to the system by setting the corresponding parameters in the functional components within the execution phase. Furthermore, the applied rule and its intended effect are recorded in the adaptation model K_{Adapt} for later purposes. If no adaptation rule is applicable or no rule approaches the system goals, we assume that the system can be set into a safe fail state. In this case, the system has to wait until (a) a suitable adaptation rule has been learned or (b) the environment changed in such a way that the goal is approached again or another rule is applicable. Both, the evaluation and the learning component are invoked by components of the MAPE-K loop but can run concurrently afterwards.

In the following, we first describe the abstract knowledge models and afterwards our two newly proposed components, the evaluation and learning component, in more detail.

A. Knowledge Models

In the last subsection, we have described the general adaptation process within our proposed reference model. We have especially sketched the usage of our proposed knowledge models. In the following, we go into details concerning features that our proposed knowledge models should have.

Environment Model K_{Env} and System Model K_{Sys}: The environment model and the system model are used as abstract representation of current relevant information about the environment and the system. This data should be as simple as possible because it is continuously collected and updated in the monitoring phase. These models could, for example, be represented by a set of sensor values and system parameters. To represent system and environment behaviour over time, some kind of history of the collected data might be necessary. Changes in the system topology are also captured in the abstract system model K_{Sys}.

Goal Model K_{Goal}: We assume an abstract goal model to comprise subgoals that describe the actual intent of the system. Subgoals can represent functional or non-functional

requirements. The goal model should allow for the judgement whether K_{Sys} together with K_{Env} satisfies the subgoals. Furthermore, it should allow for the quantification how close the system with the environment is to the goals. To this end, we assume a distance function $dist(K_{Sys}, K_{Env}, K_{Goal})$ which takes as input an abstract system and environment model, as well as the violated goal and measures the distance between the system and the environment state towards the goal. If goals are contradictory, we expect the distance function to handle this. Based on a threshold that allows the system to deviate from the goal to a certain degree, adaptation rules should be applied with the intention to approach the violated goals again.

Adaptation Rules K_{Adapt}: A crucial model of the knowledge base is the set of adaptation rules. An adaptation rule consists of three parts: the precondition, the data manipulation command, and a postcondition. The precondition (also called guard) describes when the adaptation rule is actually applicable. The precondition is evaluated on the current environment model and system model. If it evaluates to true, the corresponding adaptation rule is applicable. The command describes how the control data is manipulated when applying the rule. Finally, the postcondition describes the intended effect, i.e., how the environment (behaviour) should be influenced after applying the rule. This latter information is particularly used in the evaluation component to check whether previous adaptations were successful.

If more than one rule is applicable, the best rule w.r.t. the violated goal is chosen. To choose the best rule, we use the distance function from the goal model as described above. More precisely, we measure the distance of the current system K_{Sys} and environment model K_{Env} from the goal and compare it with the distance of the adapted system model K'_{Sys} and expected environment model K'_{Env} that would be produced by applying the respective rule. The rule that reduces the distance between the updated system and the goal the most is considered the best solution.

Above, we have discussed the different models of our proposed knowledge base. As briefly sketched, our MAPE-K loop makes use of two additional components, an evaluation component and a learning component. In the following subsections, we go into their details.

B. Evaluation component

The analysis and plan phases operate on abstract models of the system and the environment. These abstract models are continuously updated to represent the actual current system and environment state. The adaptation rules assume a certain relationship between system actions and their effect on the environment. However, this relationship could be incomplete or could change over time.

To overcome this problem, we propose an additional evaluation component which compares the intended effect of previously executed rules on the environment (recorded in the adaptation model K_{Adapt}) against the current system model K_{Sys} and environment model K_{Env}. If (some of) the previously executed rules do not have the intended effect, corresponding rules can be disabled or deleted. This is necessary, to avoid discrepancies between future model predictions and the actual system. Note that a certain tolerance could be given as part of the goal model K_{Goal} to cope with the fact that there is usually a certain gap between the actual behaviour and the model level.

Furthermore, the evaluation component is used to handle dynamic topology changes, which are captured in the system model K_{Sys}. In case of the removal of a component, adaptation rules that include actions on the removed component are disabled or deleted. In case of a newly added component, the need to update the rules due to new functionality provided by this component is detected. This need is recorded in the adaptation model K_{Adapt} and triggers the planning phase to invoke the learning component.

In summary, the evaluation component detects insufficient or missing adaptation rules. Together with the learning component, the evaluation component enables the correct co-evolution of the knowledge models and the actual system state.

C. Learning Component

In Subsection III-A, we have presented the different knowledge models that are used in our extended MAPE-K loop. While they allow for a structured adaptation process already, the adaptation process can be made adaptive itself by considering more concrete models, which we call run-time models. These models have to be executable, capture the interplay between the modelled system component and the environment, and therefore allow for a prediction of future system behaviour. We propose to include a learning component into the MAPE-K loop that is able to generate new adaptation rules at run-time using these models. It is invoked if in the planning phase no suitable adaptation rule could be applied or some dynamic topology change was detected in the evaluation phase as described in the previous section.

The learning component uses simplified reinforcement learning techniques and operates on (possibly formal) run-time models (RTMs) that represent different views on the actual system implementation. First, it updates all RTMs with the current system state K_{Sys} and environment state K_{Env}. Then, it tries to mutate the run-time models. As they represent different views and focus on different aspects, not all models have to be mutated. The relevant models are chosen according to heuristics w.r.t. the violated subgoals. The heuristics are also a parameter of the learning component. For the mutation, atomic mutation rules, which are specific to each run-time model, are applied and it is checked whether the mutated model makes an improvement towards the abstract goal model. This rating is one of the most crucial parts in our approach. It makes it necessary to quantify the quality of the run-time model w.r.t. the subgoals. To this end, a similar distance function as described in Subsection III-A is used. If the resulting mutated model is closer to the goal model, the (sequence of) mutation(s) is used to generate an adaptation rule that consists of a procedure how to change parameters of the

system. As K_{Sys} is assumed to work on similar parameters, this rule can be inserted into the knowledge model K_{Adapt}.

The learning component also plays a crucial rule in the handling of dynamic topology changes. In this case, new components have to provide their own run-time models as well as corresponding mutation rules. Then, new rules can be generated to take advantage of the newly available functionality. In this paper, we assume that the goals are disjunct and modularly applicable to one run-time model at a time.

The choice of the run-time models is crucial for the flexibility of the proposed adaptation mechanisms. In the following, we sketch how the formal process calculus *Communicating Sequential Processes* (CSP) could be used for this purpose.

D. Communicating Sequential Processes as Run-Time Model

In this subsection, we describe how CSP [12] could be used as a run-time model within the learning component and how the relationship to the knowledge base can be established. In [10], we presented an approach for the modular verification of distributed adaptive real-time systems. We primarily focussed on an abstract level where we model systems using the CSP process calculus [12]. The basic structure of the considered distributed adaptive systems is a network of adaptive components, each consisting of a functional component and an adaptation component. The functional component could directly be used as a run-time model. Its basic structure is a recursive process definition with a list of parameters.

```
FC(i,d,cd) = <process definition> -> FC(i,d',cd')
```

The variables `d` and `cd` denote the functional data and the control data, respectively. Furthermore, `i` denotes the identifier of the respective component. One possibility is to build the abstract model K_{Sys} only from the actual data that is currently used and to use the above process definition as run-time model. The learning component could mutate (parts of) the control data and evaluate whether the resulting process is closer to the system goals. If this is the case, a new adaptation rule can be generated and included into the current set of rules K_{Adapt}.

To evaluate mutations of run-time models in CSP, we need a particularly structured goal process. In general, not all properties of interest like, for example, pure liveness properties can be modelled as a finite[1] CSP process. However, for many goal properties such as safety and bounded liveness properties a finite goal process can be constructed. If the goal was, for example, that after some functional event a occurred, an event b occurs subsequently within n steps the corresponding goal process could be described as follows.

```
G(n) = |~| x:diff(Events,{a}) @ x -> G(n)
       |~| a -> G'(n)
G'(0) = b -> G(n)
G'(n) = |~| x:diff(Events,{b}) -> G'(n-1)
        |~| b -> G(n)
```

This process description says that after some a occurred, the process keeps track of the number of subsequent steps until b is performed. Furthermore, the process ensures that b must be performed after n steps at the latest.

Based on such a bounded liveness property, we can easily introduce a metric on processes describing how good they are w.r.t. the goal. To this end, the notion of CSP refinement ($\sqsubseteq$) can be used where $A \sqsubseteq B$ means that process A is refined by process B. Some process P is "better" than process Q w.r.t. some goal if it satisfies the goal for some m, i.e., $G(m) \sqsubseteq P$ and that there is no $m' < m$ such that $G(m') \sqsubseteq Q$.

Analysis of Run-Time Models: As we represent run-time models using some possibly formal specification language, we can use these models for formal analysis w.r.t. their correctness. Such an analysis is actually already included into the model to a certain degree: When mutating the run-time models, we quantify the distance between the system and the system goals. A correctness specification can be included into the goal model and be checked whenever the model was mutated to generate a new adaptation rule. To this end, it is necessary to make the set of adaptation rules a parameter of the system model. This set can be updated at run-time after rules have been removed or generated. Note that it is necessary to check the specification at run-time because in realistic applications, it is not possible to check all scenarios (in presence of some arbitrary environment), all possible mutations, and resulting adaptation rules at design time. To enable run-time verification, it is necessary that the models are as small as possible and that verification can be performed in a modular way. Ideally, verification should be incremental as well so that parts of previously achieved verification results can be reused.

Above, we have considered CSP to model the system and possibly the environment on an abstract level. CSP turns out to be a good candidate for run-time verification as well. In [10], we have shown how to verify adaptive systems in a modular fashion using refinement- and abstraction based verification. This modular verification approach together with abstraction is even more striking in run-time verification, because checking the entire system on a detailed level is not realistic.

IV. Example

In this section, we present a simplified adaptive temperature controller, which could be installed in a smart home, to illustrate our approach. The system consists of three functional components: a window controller, a heating system, and an electrical blind controller. The goal of the entire system is to keep the indoor temperature close to a predefined desired temperature w.r.t. a tolerable deviation δ (e.g., 2°C). The desired temperature can differ for day- and nighttime. The current environment state is monitored by sensors for indoor and outdoor temperature as well as for the sun intensity at the window. Furthermore, the system has a date and time module to distinguish day- and nighttime, and the seasons of year. Note that the example is primarily used to abstractly illustrate our approach. Currently, we are implementing this case study to be able to later provide experimental results.

In the following, we illustrate an adaptive design of the temperature controller following our reference model described

[1]Finiteness is needed to apply the FDR3 [13] refinement checker for CSP.

in Section III. To this end, we first explain the instantiation of the knowledge models. Then, we describe the adaptation process in situations in which the statically given set of adaptation rules cannot be applied any more. To show the variety of situations our approach can cope with, we explain what happens if a solution can be found by our proposed learning component and what happens if no solution can be found. Finally, we explain how our approach could cope with topology changes due to the inclusion of a new component.

A. Instantiation of the Knowledge Models

System Model K_{Sys}: The abstract system model contains information about the current system state, which is captured in the current parameter values of the functional components. In this example, these values are the state of the window (open, closed), the degree to which the blind is opened (0-100 %), and the flow temperature $temp_{flow}$ of the heating system. Additionally, changes in the system topology have to be captured in K_{Sys}. Therefore, we introduce three sets of components, one for currently active components, one for newly added components and one for recently disabled components. In our example setting, components can be disabled due to a defect or due to maintenance purposes. Furthermore, a component can be enabled or added if a previously defect component has been fixed, the maintenance has been completed, or if a new component has been installed.

Environment Model K_{Env}: The abstract environment model represents the current environment behaviour as captured by the sensors. Hence, K_{Env} contains the current sensor values of indoor and outdoor temperature ($temp_{in}$ and $temp_{out}$), the sun intensity at the window, and date and time information. As these values represent only a current state, some history of values, e.g., temperature values, has to be saved to represent the behaviour over time, e.g., the temperature trend.

Goal Model K_{Goal}: The abstract goal model consists of the desired daytime temperature $temp_D$, the desired nighttime temperature $temp_N$ and the tolerable deviation δ. The distance between system state and goal is given by the deviation between current and desired indoor temperature. Note that this example application gets more interesting (and more complex) if a subgoal targeting the energy consumption is added. Then, in the case of more than one applicable rule, the energy consumption of different commands can be compared and the result can be used to choose the best rule. In our example this could be used to choose between rule #2 and rule #3 to decrease the indoor temperature.

Adaptation Model K_{Adapt}: The adaptation model consists of adaptation rules (with pre and post conditions and a set of commands), some history of applied adaptation rules, and a set of disabled rules. Some possible adaptation rules for our example setting are shown in Table I. Rule #1, for example, describes that (a) on a winter day where (b) the indoor temperature is lower than the desired daytime temperature, (c) the flow temperature is not at its maximum, (d) the window is closed, (e) the sun intensity is high, and (f) the blinds are open, a possible operation to increase the indoor temperature is to raise the flow temperature by $5°$C. Hence, in the next monitoring phase the indoor temperature is expected to increase by at least $3°$C.

TABLE I
EXAMPLE ADAPTATION RULES

#	pre	commands	post
1	(a) winter(), daytime(clock), (b) $temp_{in} < temp_D - \delta$, (c) $temp_{flow} < MAX_{flow}$, (d) window.closed(), (e) sensor.sunIntensity() $> 70\%$, (f) blinds.open() = 100 %	$temp_{flow} := temp_{flow} + 5°$C	$temp'_{in} \geq temp_{in} + 3°$C
2	$temp_{in} > temp_D + \delta$, $temp_{out} < temp_{in}$	window. openWindow()	$temp'_{in}$ = calc_ TempIn($temp_{out}$, window.size())
3	$temp_{in} > temp_D + \delta$, $temp_{flow} > MIN_{flow} + 5°$C	$temp_{flow} := temp_{flow} - 5°$C	$temp'_{in} = temp_{in} - 3°$C

B. Examples of Unforeseen Situations

In this subsection, we describe two possible example situations that might not have been considered at design time. The adaptive temperature controller copes with the first situation by learning a new rule. In the second situation, no rule can be inferred and, hence, the system has to wait in a safe fail state.

First Situation – Successful Learning: As depicted in our example adaptation rule #1 in Table I, the indoor temperature is controlled by the flow temperature and the expected environment behaviour is based on an assumed relation between both temperatures. If this relationship is wrong, e.g., in case of a damaged window or a very low indoor temperature as result of a cold night without heating, the monitored environment state differs from the expected state. In this case, our evaluation component detects this gap and disables the rule. Afterwards, in the planning phase, no applicable rule is found. Hence, our learning component is invoked. Here, the affected run-time models of the heating system and the environment are updated with the current values and the history. Now, the learning phase starts and the step size of the flow temperature in the run-time model is raised according to a corresponding mutation rule. The interplay between heating system and environment model is evaluated and a new rule with a greater step size (e.g., $temp_{flow} + 7°$C) is generated and added to the knowledge base. This rule can be used in the next planning phase. Whether it leads to the desired system state will then be evaluated in the following MAPE cycle.

Second Situation – Unsuccessful Learning: In our example, the only possibilities to decrease the indoor temperature are to open the window or to lower the flow temperature (this is represented in rules #2 and #3 in Table I). The first rule (#2) is, however, only applicable if the outdoor temperature is lower than the indoor temperature. The second rule (#3) is only applicable if the heating is on. Hence, the system does not cope with hot summer days. In such a situation, the system detects an intolerable deviation from the desired indoor temperature (measured by the distance function and compared to a given threshold δ (e.g., $2°$C)) but neither finds an applicable rule in the set of adaptation rules nor by learning, as there exists no

suitable functional component. Therefore, it goes into a safe fail state until the temperature drops in the evening.

C. Example of a Topology Change

In the second situation above, a new component that provides a cooling operation is needed to fulfil the goal. If, for example, an air conditioner is installed, it is detected during monitoring (still being done in the fail state) and recorded as a newly added component within K_{Sys}. Then, the evaluation component identifies the need for an update of the adaptation rules and triggers the planning component to invoke the learning. The learning component takes the run-time model that is provided by the air conditioner, examines its capabilities in the interplay with the run-time model of the environment (updated with the current K_{Env}) and generates new rules. These can be later used in the planning phase to leave the fail state.

In this section, we have illustrated how our approach can be used to design an example application that is able to adapt to unforeseen environment behaviours, as far as its capabilities permit, as well as to changes in the system topology.

V. Conclusion

In this paper, we have presented a reference model for the design of self-adaptive systems that can dynamically adapt their adaptation mechanisms. Our approach extends IBM's MAPE-K loop. We have proposed an evaluation component that continuously evaluates former adaptation steps as well as the validity of adaptation rules in case of dynamic topology changes, and disables inefficient or invalid rules. Additionally, we have introduced a learning component that infers new adaptation rules by using online learning on run-time models. These two components together with our imposed structure on the knowledge base allow us to not only adapt cyber-physical systems by using a fixed adaptation mechanism but to make the adaptation mechanisms themselves adaptive. To illustrate the applicability of our model, we have described a possible example application in the context of a CPS for smart homes.

Currently, we are implementing the example presented in Section IV in SystemC [11]. This will allow us to provide experimental data from a running environment and to evaluate the applicability of our approach. Additionally, it forms the basis to investigate further evaluation and learning mechanisms, also in the context of more complex case studies.

In future work, we aim at alleviating the assumptions we have made in this paper. For example, we would like to cope with dynamic goals, external rating of adaptations, and more sophisticated run-time models.

Currently, we assume the structure of run-time models to be fixed. To even better cope with unforeseen environment changes, it would be beneficial to continuously learn the environment behaviour and to appropriately update a run-time model of the environment, also structurally. This would allow for a more precise prediction of the effect of mutations.

In the discussion of analysis possibilities of formal run-time models, we have primarily focussed on approaching a certain goal. It would also be possible to check safety and liveness properties at run-time. It is an important question for future work, how this can be performed efficiently. We think that we are able to apply our previously presented abstraction and refinement techniques [10] for this purpose.

In this paper, we have assumed that only one run-time model is used at a time. In future work, we would also like to combine information from several run-time models. This is particularly important to infer new adaptation rules that influence several (distributed) parts of the system at the same time.

Finally, it would be interesting to not only adapt the adaptation process, but also mutation rules that are applied to the model. By following this basic principle, it would be possible to establish a higher-order adaptation approach where adaptation can take place on arbitrary levels of abstraction.

References

[1] IBM Corp., *An architectural blueprint for autonomic computing*. USA: IBM Corp., Oct. 2004.

[2] K. Schneider, T. Schuele, and M. Trapp, "Verifying the adaptation behavior of embedded systems," in *Proceedings of the 2006 international workshop on Self-adaptation and self-managing systems*, ser. SEAMS '06. New York, NY, USA: ACM, 2006, pp. 16–22.

[3] R. Adler, I. Schaefer, T. Schüle, and E. Vecchié, "From model-based design to formal verification of adaptive embedded systems," in *Formal Methods and Software Engineering, 9th International Conference on Formal Engineering Methods, ICFEM 2007, Boca Raton, FL, USA, November 14-15, 2007, Proceedings*, 2007, pp. 76–95.

[4] S. Jaskó, G. Simon, K. Tarnay, T. Dulai, and D. Muhi, "CSP-based modelling for self-adaptive applications," in *Infocommunications Journal*, vol. LVIV, no. 2009/II, 2009, pp. 14–21.

[5] M. Luckey and G. Engels, "High-quality specification of self-adaptive software systems," in *Proceedings of the 8th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*, ser. SEAMS '13. Piscataway, NJ, USA: IEEE Press, 2013, pp. 143–152.

[6] A. Bennaceur, R. France, G. Tamburrelli, T. Vogel, P. Mosterman, W. Cazzola, F. Costa, A. Pierantonio, M. Tichy, M. Akşit, P. Emmanuelson, H. Gang, N. Georgantas, and D. Redlich, "Mechanisms for leveraging models at runtime in self-adaptive software," in *Models@run.time*, ser. Lecture Notes in Computer Science, N. Bencomo, R. France, B. Cheng, and U. Aßmann, Eds. Springer, 2014, vol. 8378, pp. 19–46.

[7] U. Aßmann, S. Götz, J.-M. Jézéquel, B. Morin, and M. Trapp, "A reference architecture and roadmap for models@run.time systems," in *Models@run.time*. Springer, 2014, pp. 1–18.

[8] M. U. Iftikhar and D. Weyns, "Activforms: Active formal models for self-adaptation," in *Proceedings of the 9th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*, ser. SEAMS 2014. New York, NY, USA: ACM, 2014, pp. 125–134.

[9] J. Bengtsson and W. Yi, "Timed automata: Semantics, algorithms and tools," in *Lectures on Concurrency and Preti Nets*. Springer, 2004, pp. 87–124.

[10] T. Göthel, V. Klös, and B. Bartels, "Modular design and verification of distributed adaptive real-time systems based on refinements and abstractions," *EAI Endorsed Transactions on Self-Adaptive Systems*, vol. 15, no. 1, 2015.

[11] IEEE Standards Association, "IEEE Std. 1666–2005, Open SystemC Language Reference Manual," 2005.

[12] S. Schneider, *Concurrent and Real Time Systems: The CSP Approach*. New York, NY, USA: John Wiley & Sons, Inc., 1999.

[13] T. Gibson-Robinson, P. Armstrong, A. Boulgakov, and A. Roscoe, "FDR3 — a modern refinement checker for CSP," in *Tools and Algorithms for the Construction and Analysis of Systems*, ser. Lecture Notes in Computer Science, E. Ábrahám and K. Havelund, Eds. Springer, 2014, vol. 8413, pp. 187–201.